

Konfiguration und Policies

Konvergenz-Policies

Die Konvergenz-Policy entscheidet nach jeder Evaluationsrunde, wie der Loop fortgesetzt wird.

DefaultConvergencePolicy

```
var pipeline = Geef.CreatePipeline<string>()
    // ...
    .UseConvergencePolicy(new DefaultConvergencePolicy
    {
        MaxIterations = 15,                // Max. 15 Versuche (Default: 10)
        MaxElapsedTime = TimeSpan.FromMinutes(45), // Max. 45 min (Default: 30 min)
        StagnationThreshold = 5,           // Stagnation nach 5 gleichen Runden (Default: 3)
        AbortOnCritical = true,            // Bei Critical-Finding sofort abbrechen (Default: true)
        DetectRegression = true            // Regression erkennen (Default: true)
    })
    .Build();
```

Entscheidungslogik

Die Policy prüft in dieser Reihenfolge:

1. **Approved?** — Wenn `IsFullyApproved`, dann `ConvergenceDecision.Approved`
2. **Critical?** — Wenn `AbortOnCritical` und ein Finding mit `Severity.Critical` existiert: `AbortCriticalBlocker`
3. **Max Zeit?** — Wenn `elapsed > MaxElapsedTime`: `StopMaxAttemptsReached`
4. **Max Iterationen?** — Wenn `history.Count >= MaxIterations`: `StopMaxAttemptsReached`
5. **Stagnation?** — Wenn `history.IsStagnant(StagnationThreshold)`: `StopStagnant`
6. **Regression?** — Wenn `DetectRegression` und `history.HasRegression()`: `StopRegression`
7. **Sonst:** `Continue` — weiter iterieren

Eigene Konvergenz-Policy

```
public class CostAwareConvergencePolicy : IConvergencePolicy
{
    public decimal MaxBudget { get; init; } = 10.0m;

    public ConvergenceDecision Evaluate(
        IterationHistory history,
        EvaluationAggregate currentAggregate,
        TimeSpan elapsed)
    {
        if (currentAggregate.IsFullyApproved)
            return ConvergenceDecision.Approved;

        // Eigene Logik: Budget-basierte Entscheidung
        var estimatedCost = history.Count * 0.5m;
        if (estimatedCost > MaxBudget)
            return ConvergenceDecision.EscalateToHuman;

        return ConvergenceDecision.Continue;
    }
}
```

Evaluation Strategies

SequentialEvaluationStrategy (Default)

Führt Reviewer der Reihe nach aus. Deterministisch, einfach zu debuggen.

```
.UseEvaluationStrategy(new SequentialEvaluationStrategy())
```

ParallelEvaluationStrategy

Alle Reviewer gleichzeitig. Schnellste Gesamtausführung, aber alle laufen immer vollständig.

```
.UseEvaluationStrategy(new ParallelEvaluationStrategy())
```

FailFastEvaluationStrategy

Parallel, aber sofortige Rückkehr nach erstem `Rejected` oder `Failed`. Spart Kosten bei teuren KI-Reviews.

```
.UseEvaluationStrategy(new FailFastEvaluationStrategy())
```

Verhalten:

- Alle Reviewer starten gleichzeitig
- Sobald ein Reviewer `Rejected` oder `Failed` zurückgibt, wird das CancellationToken für die anderen Reviewer ausgelöst
- Die Methode kehrt sofort zurück — verbleibende Tasks werden nicht blockierend abgewartet
- Ausnahmen verbleibender Tasks werden non-blocking beobachtet

PriorityOrderedEvaluationStrategy

Sequentiell nach Priorität (niedrigerer Wert = höhere Priorität). Bricht ab, sobald ein Reviewer mit einem Error-Finding rejected.

```
.UseEvaluationStrategy(new PriorityOrderedEvaluationStrategy())
```

Anwendungsfall: Günstige Syntax-Checks (Priorität 10) vor teuren KI-Reviews (Priorität 100):

```
public class SyntaxChecker : IReviewer
{
    public string Name => "SyntaxCheck";
    public int Priority => 10; // Läuft zuerst
    // ...
}

public class AiCodeReview : IReviewer
{
    public string Name => "GPT-Review";
    public int Priority => 100; // Läuft nur, wenn Syntax OK
    // ...
}
```

Middleware

Middleware registrieren

```
var pipeline = Geef.CreatePipeline<string>()
    // ...
    .UseMiddleware(new TimeoutMiddleware
    {
        DefaultTimeout = TimeSpan.FromMinutes(2),
        PhaseTimeouts = new()
        {
            [GeefPhase.Execution] = TimeSpan.FromMinutes(5),
            [GeefPhase.Evaluation] = TimeSpan.FromMinutes(3)
        }
    })
    .UseMiddleware(new TracingMiddleware())
    .UseMiddleware(new ExceptionHandlingMiddleware())
    .Build();
```

Reihenfolge: Middleware wird in der Registrierungsreihenfolge ausgeführt — die zuerst registrierte Middleware ist die äußerste im Chain.

TimeoutMiddleware

Erzwingt Phase-spezifische Timeouts. Erstellt ein verknüpftes Cancellation-Token, das nach Ablauf der Zeit ausgelöst wird.

```
new TimeoutMiddleware
{
    DefaultTimeout = TimeSpan.FromMinutes(5),
    PhaseTimeouts = new Dictionary<GeefPhase, TimeSpan>
    {
        [GeefPhase.Grounding] = TimeSpan.FromMinutes(1),
        [GeefPhase.Execution] = TimeSpan.FromMinutes(10),
        [GeefPhase.Evaluation] = TimeSpan.FromMinutes(3),
        [GeefPhase.Finalize] = TimeSpan.FromSeconds(30)
    }
}
```

Bei Timeout wird `PhaseTimeoutException` geworfen (mit `Phase` und `Timeout` Properties).

TracingMiddleware

Erstellt OpenTelemetry-Spans für jede Phase:

- Span-Name: `geef.middleware.{Phase}`
- Tags: `geef.phase`, `geef.run_id`, `geef.iteration`

ExceptionHandlerMiddleware

Wandelt unerwartete Exceptions in `ProviderException` um und setzt `Phase` aus dem Middleware-Context sowie `ProviderName` aus `GeefMiddlewareContext.ComponentName`. Hinweis: `ComponentName` wird nur dann befüllt, wenn der Aufrufer es im Context setzt — der Standard-Runner setzt es nicht, da er Exceptions direkt vor dem Middleware-Aufruf abfängt.

Eigene Middleware schreiben

```
public class RetryMiddleware : IGeefMiddleware
{
    public int MaxRetries { get; init; } = 3;

    public async Task InvokeAsync(GeefMiddlewareContext ctx, Func<Task> next)
    {
        for (var attempt = 1; attempt <= MaxRetries; attempt++)
        {
            try
            {
                await next();
                return;
            }
            catch (Exception) when (attempt < MaxRetries)
            {
                await Task.Delay(TimeSpan.FromSeconds(attempt * 2));
            }
        }
    }
}
```

Mehrere Reviewer

```
var pipeline = Geef.CreatePipeline<CodeResult>()
    .UseGrounding(new RepoGrounding())
    .UseExecution(new CodeGenerator())
    .AddReviewer(new SyntaxReviewer())           // Check 1: Kompiliert der Code?
    .AddReviewer(new SecurityReviewer())         // Check 2: Sicherheitslücken?
    .AddReviewer(new StyleReviewer())            // Check 3: Code-Style?
    .AddReviewer(new TestCoverageReviewer())     // Check 4: Testabdeckung?
    .UseFinalizer(new CodeFinalizer())
    .UseEvaluationStrategy(new PriorityOrderedEvaluationStrategy())
    .Build();
```

Event-Konfiguration

Inline-Delegates

```
.ConfigureEvents(e =>
{
    e.OnPipelineStarted = evt =>
    {
        Console.WriteLine($"[{evt.RunId}] Pipeline gestartet mit Input: {evt.Input}");
        return Task.CompletedTask;
    };

    e.OnEvaluationRejected = evt =>
    {
        Console.WriteLine($"[{evt.RunId}] Iteration {evt.Iteration}: Abgelehnt ({evt.Decision})");
        return Task.CompletedTask;
    };

    e.OnPipelineCompleted = evt =>
    {
        Console.WriteLine($"[{evt.RunId}] Fertig in {evt.TotalIterations} Iterationen ({evt.TotalDuration})");
        return Task.CompletedTask;
    };
})
```

Logging-Sink

```
using Microsoft.Extensions.Logging;

var loggerFactory = LoggerFactory.Create(b => b.AddConsole());
var logger = loggerFactory.CreateLogger("Geef");

.AddEventSink(new LoggingEventSink(logger))
```

Mehrere Sinks

```
.AddEventSink(new LoggingEventSink(logger))  
    .ConfigureEvents(e =>  
    {  
        e.OnPipelineFailed = evt => SendAlertAsync(evt);  
    })
```

Alle registrierten Sinks werden über einen internen `CompositeEventSink` sequentiell bedient.